

Builder Lab Guide: Lakebase Day 1

For: engineers building in the room. **When:** Day 1 of the session. **Workspace:** your Databricks workspace (<https://<your-workspace>.cloud.databricks.com>). **Catalog:** the workshop catalog your facilitator gives you.

You are here to build a **Field Medical Copilot** on Lakebase with your own hands. The morning is fundamentals and the agent-memory build; the afternoon closes with two guided demos (Search and the Online Feature Store). This guide is your day-of checklist: what you set, what you run, and what you should see at each step. If a step goes red, the fix is almost always in the troubleshooting table at the end.

You are not setting up infrastructure. A platform owner pre-provisioned the shared project, the synthetic data, the Search demo project, and the feature-store endpoint before you sat down. You arrive to a warm workspace and start building at Module 1. If Module 1 cannot connect, raise your hand: that is a room-wide setup issue, not something you fix at your keyboard.

The model: what is shared, what is yours

One decision shapes the whole day, so understand it before you start.

Resource	Shared or yours	Why
Lakebase project (<code>az-workshop-copilot</code>)	Shared	Creating a project is slow and admin-gated. You get <code>CAN_CONNECT_AND_CREATE</code> on the shared one.
Your schema (<code>copilot_<your-id></code>)	Yours	You create and own it. All your tables live here. Nobody else writes to it.
Synthetic dataset (Unity Catalog)	Shared, read-only	One generated copy of the pharma data. You read it; you never modify it.
Search demo project (<code>az-demo-search</code>)	Shared, demo-only	Enabling Search is a one-way door that restarts compute. Done once, on a throwaway project. You do not enable it.
Online feature store + serving endpoint	Shared, demo-only	Provisioning takes far too long to do live. Pre-built. You watch it score.

The takeaway: **you build agent memory hands-on in your own schema. Search and the feature store are guided demos you follow along with.** That split is deliberate. Memory is where the mechanics are the lesson, so you build it. Search and the feature store are where the provisioning is just plumbing, so we pre-build and drive them together.

Pre-flight (do this once, first 5 minutes)

1. Confirm you can reach the workspace

Open `https://<your-workspace>.cloud.databricks.com` in the browser on your AZ laptop. You should land in the workspace without a VPN prompt failing. If the page will not load or auth fails, that is the one blocker the platform team must clear; raise it now.

2. Clone the workshop repo into a Git folder

In the workspace: **Workspace** → **your home** → **Create** → **Git folder**, point it at the workshop repo. This puts every notebook plus `workshop/config.py` on your path. You run everything from here in the browser; nothing installs on your laptop.

3. Set your values in config.py

Every notebook reads its workspace settings from `workshop/config.py`. This is the only file you edit. Open it in your Git folder and set these values, then save.

- **The shared values your facilitator gives you.** Fill in `project_id` (the shared Lakebase project), `search_project_id` (the Search demo project), `warehouse_id`, `uc_catalog`, and `uc_schema`. Everyone in the room uses the same values for these.
- **Your own team id.** Set `team_id` to your initials (lowercase letters and digits, starts with a letter, e.g. `psk`). This becomes your private schema `copilot_<your-id>`, so nobody writes to anybody else's tables.

Edit the file on disk. Do not try to set these through environment variables in a notebook cell. The notebooks call `%restart_python` in their first cell to upgrade the SDK, and that restart clears any environment variable you set in a cell, so the value would be lost before the notebook reads it. A value saved in `config.py` survives the restart.

Then confirm what you set:

```
import sys; sys.path.insert(0, "..")
from workshop.config import cfg
cfg.show()           # prints every resolved value as a table
cfg.validate()       # fails fast and lists every problem at once, no network calls
```

`cfg.show()` should print your `app_schema` as `copilot_<your-id>`, the shared `project_id`, your workshop catalog, and the warehouse. If `validate()` lists a problem, fix that line in `config.py` before you run anything else. A green `validate()` is your gate.

4. Run the environment validator

Open `workshop/00_validate_environment.py` and run it top to bottom.

- [] It prints ten-plus pass/fail rows and **runs every check even if one fails**.
- [] The **DDL canary** row is green. That proves you can create, write, read, and drop under your own identity. If it is red you have connect-but-not-create; you need `USAGE` + `CREATE` on the schema, so raise your hand.
- [] The final line does not raise. If it raises, one row above is red; read which one.

Green validator = you are cleared to build. This is the single most important checkpoint of the morning.

Module 1: Fundamentals (hands-on, ~20 min)

Notebook: `01_fundamentals.py`

What you do: connect to Lakebase over the notebook connection and see the hierarchy for real (project, branch, endpoint, database). This sets the mental model for everything after.

- [] The notebook installs `databricks-sdk>=0.81.0` and restarts Python in its first cell. **Let the restart finish before running the next cell.** The runtime ships an older SDK that lacks `w.postgres` ; this line is what fixes it.
- [] You see the **Postgres version** printed over the notebook connection (PG 17).
- [] You understand the four-level hierarchy: Project → Branch → Endpoint → Database.

You should see: a Postgres 17 version string returned through your own connection. That is Lakebase answering you directly.

Module 2: Synced tables (walk-through, ~15 min)

Notebook: `02_synced_tables.py`

Synced tables mirror a Unity Catalog Delta table into Postgres. The platform owner created these via the Catalog UI before the room (the CLI path is broken on the current release; the UI is the reliable path). You walk the created table and learn the one lesson that surprises people.

- [] You can read the mirrored `hcp_interactions` rows from Postgres.
- [] You understand the governance lesson: on Autoscaling Lakebase, a synced table is a **writable** partitioned Postgres table. It is **not** read-only at the engine. Read-only is a **governance rule you enforce with GRANTS**, not something the engine guarantees.

You should see: the same interaction rows in Postgres that exist in the Delta source, and the demo showing a write is not rejected by the engine (which is why GRANTS matter).

Module 3: Agent memory, built from scratch (hands-on, the core, ~60 min)

Notebook: `03_agent_memory_build.py`

This is the heart of the day, and it maps directly to what AstraZeneca already chose Lakebase for: agent logs and short-term memory. You build working memory in four moves, each motivated by the failure of the one before.

- [] **Move 1, sessions and turns.** You create the schema and tables in **your own** `copilot_<your-id>` . You see that a **committed** write is visible to a brand-new connection, and an **uncommitted**

write is not. This is the ACID guarantee a separate vector store cannot make.

- [] **Move 2, recall.** You see that pure recency buries the pertinent turn, and that ranking by relevance first (topic match, then recency) surfaces it.
- [] **Move 3, durable facts.** You create a facts table distinct from raw turns, with provenance back to the source turns and a status so a fact can be a *candidate* rather than truth.
- [] **Move 4, scope isolation (the security lesson).** You prove that the rep filter belongs **inside** the query that reads the rows, not in an outer filter. The notebook shows the leak (an outer filter with a bug) and the correct version side by side. The assertion requires a non-empty result of only your rep's rows.
- [] You hand-promote one turn into a durable fact with provenance, then ask the obvious question the next module answers: *what happens at hundreds of turns across a dozen reps?*

You should see: recall returning this rep's turns and **provably not** another rep's, and one durable fact joined to its source turn. The final cell prints `agent memory works: committed, recallable, scope-isolated, with provenance`.

If you finish early: read the "Contrast: managed agent memory" cell at the end. It explains when to reach for managed agent memory (a Unity Catalog securable, not Lakebase-backed) instead of building your own. The framing: *Lakebase when you want the SQL, managed when you don't*.

Module 3b: Dreaming: offline memory consolidation (hands-on, ~40 min)

Notebook: `03b_agent_memory_dreaming.py`

You just promoted one fact by hand. Now you run the process that does it at scale while nobody is talking to the agent: **consolidate, abstract, decay/reconcile**, run as one job. Every derived fact is a candidate, not truth, with a confidence gate.

- [] The ingest cell loads the seeded turns into your schema and asserts you have **700+** of them. Re-running does not duplicate (the source turn id carries a unique constraint).
- [] **Consolidate:** turns grouped **within the rep-and-HCP scope** become candidate facts with provenance. Scope isolation survives dreaming; one rep's turns never fold into another rep's fact.
- [] **Abstract:** a cross-rep theme no single conversation contains surfaces as an org-scoped fact (the Cardiology segment repeatedly raising reimbursement).
- [] **The review gate, the point of the module.** A planted, plausible-but-wrong clinical claim (a CYP3A4 interaction claim) lands in the **review queue**, invisible to the agent. The assertion checks the claim is **held in review**, not silently dropped, and that the agent can retrieve it **zero** times.

You should see: the org-scoped Cardiology reimbursement fact with provenance to its source turns, and the line the review gate holds: `claim captured for review, invisible to the agent`. That is the safety property that makes memory usable in a regulated field-medical context.

Module 4: Hybrid Search (guided demo, ~20 min)

Notebook: `04_search_hybrid.py`

Run against the shared, pre-enabled `az-demo-search` project. You follow along; you do **not** enable Search yourself (the enable is irreversible and restarts compute for everyone on the project).

- [] You see a **hybrid** query combining vector similarity and keyword search, fused with RRF, in a **single** query.
- [] `EXPLAIN` shows the index in use.
- [] You understand the copilot mapping: "find the approved, in-date content that answers this HCP's question" by meaning and keyword together.

You should see: ranked hybrid results and an `EXPLAIN` plan that confirms the index is doing the work, not a sequential scan.

Module 5: Online Feature Store (guided demo, ~20 min)

Notebook: `05_online_feature_store.py`

Run against the pre-provisioned online store and serving endpoint. The notebook is **fail-closed**: it checks the endpoint is READY before it does anything, so if the store was not pre-provisioned it stops cleanly rather than hanging.

- [] You send a **keys-only** request (just the HCP id) and the endpoint returns a scored prediction.
- [] You understand why this is pre-provisioned: live `create_online_store` is far too slow for a room. Sub-one-minute serving is AstraZeneca's stated requirement and the tier this builds toward.

You should see: a keys-only request returning a scored next-best-action prediction from fresh features, served in one call.

If you fall behind

The morning build (Modules 1, 3, 3b) is the part to protect. If you are behind:

- **Skip Module 2's detail.** Read the one governance lesson (synced tables are writable; enforce read-only with GRANTS) and move on.
 - **Modules 4 and 5 are demos anyway.** If you did not run them yourself, you lost nothing: watch the driven version. The code is yours to run later.
 - **Do not skip Module 3.** If you only do one thing hands-on, it is the agent-memory build. That is the capability AZ already committed to and the reason you are in the room.
 - **Everything is replayable.** You keep the repo and your schema. See "Take it home" below.
-

Quick troubleshooting

Symptom	Fix
'WorkspaceClient' object has no attribute 'postgres'	Runtime SDK too old. Let the first cell's <code>%pip install "databricks-sdk>=0.81.0"</code> + <code>%restart_python</code> finish before running on.
no unique or exclusion constraint matching the ON CONFLICT	Already fixed in the notebooks (full unique index). If you see it, you are on an old copy; re-pull the repo.
permission denied for schema (SQLSTATE 42501)	Connect-but-not-create. You need <code>USAGE</code> + <code>CREATE</code> on the schema. Raise your hand.
Workshop config is not ready ...	Run <code>cfg.validate()</code> and fix every listed item. Usually <code>WS_TEAM_ID</code> is unset.
Connection refused right after idle	Scale-to-zero woke the endpoint. Just retry; wake is about 100 ms.
Token expired after about an hour	OAuth database credentials expire after 1 hour. Re-run the connection cell to mint a fresh one.
A write to a synced table succeeded	Expected on Autoscaling. Read-only is a GRANT rule, not an engine guarantee (Module 2 lesson).

Fuller detail lives in `docs/TROUBLESHOOTING.md` and `docs/RUNBOOK.md` .

Take it home

Everything you ran is portable. To reproduce this in your own AstraZeneca workspace after the session, follow `docs/RUNBOOK.md` : it starts one step earlier than today (you create the project yourself instead of arriving to a pre-provisioned one) and walks the same modules end to end. Your schema and its tables persist in the shared project until teardown, so you can revisit your build.